# Processing Command-Line Arguments to Your Code

Just as you can pass arguments to methods in your Ruby code, you can pass arguments from the command line to the Ruby script itself. Ruby provides mechanisms for capturing arguments passed to the script and allowing you to read and parse them as part of your script.

## ARGV

Any command-line arguments after the program filename are available to your Ruby program in the global array ARGV. For instance, assume test.rb contains the following program:

```
ARGV.each { |arg| p arg }
```

If you invoke it with the following command line:

```
$ ruby -w test.rb "Hello World" a1 1.6180
```

It'll generate the following output:

```
"Hello World"
"a1"
"1.6180"
```

There's a gotcha here for all you C programmers. In Ruby, ARGV[0] is the first argument to the program, not the program name. The name of the current program is available in the global variable $0, which is aliased to $PROGRAM_NAME. All the values in ARGV are strings.

If your program reads from standard input (or uses the special object ARGF, described in the next section), the arguments in ARGV will be taken to be filenames, and Ruby will read from these files. If your program takes a mixture of arguments and filenames, make sure you empty the nonfilename arguments from the ARGV array before reading from the files.

## ARGF

It is common for a command-line program to take a list of zero or more file names to process. It will then read through these files in turn, doing whatever it does. Imagine a command that takes a list of log files and processes them line by line, like process.rb log1 log2 log3. It'd be handy to be able to treat all the log file arguments as a single logical file object.

Ruby provides a convenience object, referenced by the name ARGF, that handles access to these files, allowing you to treat the files as a single stream. The data for ARGF object is taken from the values in ARGV. The assumption is that when you use ARGF all the elements in the ARGV array are filenames. What this means is that any non filename arguments need to be removed from the ARGV array before you start reading from them using ARGF. Conversely, any filenames you add to ARGV in your code will be available to ARGF just as though they were supplied in the command line. We recommend that you do any ARGV manipulation before you start reading from ARGF.

The ARGF object defines most of the same read methods that IO does, including gets, read, and readline. If you read from ARGF, Ruby will open the file whose name is the first element of ARGV and perform the I/O on it. If, as you continue to read, you reach the end of that file, Ruby closes it, shifts it out of the ARGV array, and then opens the next file in the list. At some point, when you have finished reading from the last file, ARGF will return an end-of-file